



React.js

Day 2 - Hooks, Routing & State Management

Complete Study Notes with Examples

Topics: `useEffect` • `useRef` • `useContext` • React Router • Custom Hooks 2025 Edition

■ Table of Contents

#	Topic	Key Concepts
1	useEffect Hook	Side effects, dependency array, cleanup
2	useRef Hook	DOM refs, persisting values, focus management
3	useContext Hook	Context API, createContext, Provider, Consumer
4	useReducer Hook	Reducer pattern, dispatch, complex state
5	Custom Hooks	Extracting logic, reusability, naming rules
6	React Router v6	BrowserRouter, Routes, Route, Link, useNavigate
7	useParams & useLocation	URL params, query strings, navigation state
8	Lifting State Up	Sharing state, sibling communication
9	Performance Hooks	useMemo, useCallback, React.memo
10	Mini Project	Multi-page Todo App combining all Day 2 topics
11	Quick Reference	Hooks cheat-sheet & Router cheat-sheet

■ 1. useEffect Hook

useEffect lets you perform **side effects** in functional components - things that happen outside of rendering, such as fetching data, subscribing to events, updating the document title, or setting timers.

Syntax

useEffect Syntax

```
import { useEffect } from 'react';

useEffect(() => {
  // Side effect code here

  return () => {
    // Optional cleanup function (runs on unmount or before next effect)
  };
}, [dependency1, dependency2]); // Dependency array
```

Dependency Array - The Three Cases

Dependency Array	When Effect Runs	Use Case
No array - <code>useEffect(() => {})</code>	After EVERY render	Rarely used
Empty <code>[]</code> - <code>useEffect(() => {}, [])</code>	Once on mount only	Initial data fetch, subscriptions
<code>[val]</code> - <code>useEffect(() => {}, [val])</code>	When val changes	Sync with prop/state

Example 1 - Document Title Update

useEffect - Title Update

```
import { useState, useEffect } from 'react';

function PageTitle() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  }, [count]); // Re-run only when count changes

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

■ 2. useRef Hook

useRef returns a mutable ref object whose **.current** property is initialized to the given value. Unlike state, updating a ref does **not** cause a re-render. It has two main uses: accessing DOM elements directly, and storing mutable values that persist across renders.

Use Case 1 - Accessing DOM Elements

useRef - DOM Access

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus();    // Directly access the DOM node
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Click button to focus" />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}
```

Use Case 2 - Storing Previous Value

useRef - Previous Value

```
import { useState, useEffect, useRef } from 'react';

function PreviousValue() {
  const [count, setCount] = useState(0);
  const prevCount = useRef(0);

  useEffect(() => {
    prevCount.current = count;    // Store after every render
  });

  return (
    <div>
      <p>Current: {count} | Previous: {prevCount.current}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
}
```

Use Case 3 - Interval Timer without Re-render

useRef - Timer

```
import { useState, useEffect, useRef } from 'react';

function Stopwatch() {
```

```
const [time, setTime] = useState(0);
const timerRef = useRef(null); // Holds interval ID

const start = () => {
  timerRef.current = setInterval(() => setTime(t => t + 1), 1000);
};

const stop = () => clearInterval(timerRef.current);

return (
  <div>
    <p>Time: {time}s</p>
    <button onClick={start}>Start</button>
    <button onClick={stop}>Stop</button>
  </div>
);
}
```

■ **Tip:** Use `useRef` when you need to read/write a value without triggering a re-render.

■ 3. useContext Hook & Context API

The **Context API** solves the problem of **prop drilling** - passing props through many layers of components that don't actually need them. Context provides a way to share values (theme, auth, language) globally across the component tree.

3 Steps to Use Context

- **Step 1 - Create** the context with `React.createContext()`
- **Step 2 - Provide** a value using the `Context.Provider` wrapper
- **Step 3 - Consume** the value with `useContext()` in any child component

Full Example - Theme Context

ThemeContext.js

```
// Step 1: Create - ThemeContext.js
import { createContext, useState, useContext } from 'react';

const ThemeContext = createContext(); // Creates the context object

// Custom provider component
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');

  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

// Custom hook for easy consumption
export function useTheme() {
  return useContext(ThemeContext);
}
```

index.js - Wrap with Provider

```
// Step 2: Wrap app with Provider - index.js
import { ThemeProvider } from './ThemeContext';

root.render(
  <ThemeProvider>
    <App />
  </ThemeProvider>
);
```

Navbar.js - Consuming Context

```
// Step 3: Consume anywhere - Navbar.js
import { useTheme } from './ThemeContext';

function Navbar() {
  const { theme, toggle } = useTheme();
```

```
return (
  <nav style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
    <h1>My App</h1>
    <button onClick={toggle}>
      Switch to {theme === 'dark' ? 'Light' : 'Dark'} Mode
    </button>
  </nav>
);
}
```

Auth Context Example (Common Pattern)

AuthContext.js

```
// AuthContext.js
import { createContext, useState, useContext } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => setUser(userData);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

// Usage in any component:
// const { user, login, logout } = useAuth();
```

■ 4. useReducer Hook

useReducer is an alternative to **useState** for managing **complex state logic**. It is inspired by Redux and works well when the next state depends on the previous state, or when multiple sub-values need to be managed together.

Syntax

useReducer Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);

// reducer function signature:
function reducer(state, action) {
  switch (action.type) {
    case 'ACTION_TYPE':
      return { ...state, /* changes */ };
    default:
      return state;
  }
}
```

Counter with useReducer

useReducer - Counter

```
import { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    case 'DECREMENT': return { count: state.count - 1 };
    case 'RESET':     return initialState;
    case 'SET':       return { count: action.payload };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>
      <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
      <button onClick={() => dispatch({ type: 'SET', payload: 10 })}>Set 10</button>
    </div>
  );
}
```

Shopping Cart with useReducer

useReducer - Shopping Cart

```
const cartReducer = (state, action) => {
  switch (action.type) {
    case 'ADD_ITEM':
      const exists = state.items.find(i => i.id === action.payload.id);
      if (exists) {
        return {
          ...state,
          items: state.items.map(i =>
            i.id === action.payload.id ? { ...i, qty: i.qty + 1 } : i
          )
        };
      }
      return { ...state, items: [...state.items, { ...action.payload, qty: 1 }] };

    case 'REMOVE_ITEM':
      return { ...state, items: state.items.filter(i => i.id !== action.payload) };

    case 'CLEAR_CART':
      return { items: [] };

    default:
      return state;
  }
};

// Usage:
const [cart, dispatch] = useReducer(cartReducer, { items: [] });
dispatch({ type: 'ADD_ITEM', payload: { id: 1, name: 'Book', price: 299 } });
dispatch({ type: 'REMOVE_ITEM', payload: 1 });
```

■ ■ **Info:** Use useReducer over useState when state has 3+ sub-values or transitions depend on previous state.

■ 5. Custom Hooks

Custom hooks are plain JavaScript functions that start with **use** and can call other React hooks internally. They let you **extract and reuse stateful logic** across multiple components without changing the component hierarchy.

■ **Tip:** Naming rule: custom hooks **MUST** start with 'use' - e.g., useFetch, useLocalStorage, useToggle.

Custom Hook 1 - useToggle

useToggle Custom Hook

```
import { useState } from 'react';

function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);
  const toggle = () => setValue(v => !v);
  return [value, toggle];
}

// Usage in any component
function Modal() {
  const [isOpen, toggleModal] = useToggle(false);

  return (
    <div>
      <button onClick={toggleModal}>
        {isOpen ? 'Close' : 'Open'} Modal
      </button>
      {isOpen && <div className="modal">Modal Content</div>}
    </div>
  );
}
```

Custom Hook 2 - useFetch

useFetch Custom Hook

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => {
        if (!res.ok) throw new Error('Network error');
        return res.json();
      })
      .then(data => { setData(data); setLoading(false); })
      .catch(err => { setError(err.message); setLoading(false); });
  }, [url]);

  return { data, loading, error };
}
```

```
}

// Usage - clean and reusable!
function Posts() {
  const { data, loading, error } = useFetch('https://jsonplaceholder.typicode.com/posts');

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{data.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Custom Hook 3 - useLocalStorage

useLocalStorage Custom Hook

```
import { useState } from 'react';

function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = (value) => {
    setStoredValue(value);
    window.localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue];
}

// Usage
function Settings() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Theme: {theme}
    </button>
  );
}
```

■ ■ 6. React Router v6

React Router is the standard library for adding **client-side routing** to React apps. It allows you to build Single Page Applications (SPAs) with multiple 'pages' without full page reloads.

Installation

Terminal

```
npm install react-router-dom
```

Core Components

Component / Hook	Purpose
BrowserRouter	Wraps app; enables HTML5 history routing
Routes	Container for all Route definitions
Route	Maps a URL path to a component
Link	Navigation link (no page reload)
NavLink	Like Link but adds active class automatically
Outlet	Renders child routes inside a layout
useNavigate()	Programmatic navigation (redirect)
useParams()	Access URL parameters (:id)
useLocation()	Access current URL, pathname, state
useSearchParams()	Read and set query string parameters

Basic Router Setup

index.js - Wrap with BrowserRouter

```
// index.js
import { BrowserRouter } from 'react-router-dom';

root.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

App.js - Route Definitions

```
// App.js
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';
import NotFound from './pages/NotFound';

function App() {
  return (
```

```
<Routes>
  <Route path="/"          element={<Home />} />
  <Route path="/about"     element={<About />} />
  <Route path="/contact"   element={<Contact />} />
  <Route path="*"          element={<NotFound />} /> { /* 404 */}
</Routes>
);
}
```

Navigation with Link and NavLink

Link and NavLink

```
import { Link, NavLink } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
      { /* Link - basic navigation */}
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>

      { /* NavLink - adds 'active' class when route matches */}
      <NavLink
        to="/contact"
        style={({ isActive }) => ({ color: isActive ? 'blue' : 'black' })}
      >
        Contact
      </NavLink>
    </nav>
  );
}
```

■ ■ 10. Mini Project - Multi-page Todo App

This project combines **useReducer**, **Context**, **React Router**, and **custom hooks** into a multi-page Todo application.

File Structure

Project Structure

```
src/
|-- context/
|   |-- TodoContext.js    <= useReducer + Context
|-- hooks/
|   |-- useTodos.js       <= Custom hook
|-- pages/
|   |-- Home.js           <= Todo list page
|   |-- About.js          <= About page
|-- components/
|   |-- Navbar.js
|   |-- TodoForm.js
|   |-- TodoItem.js
|-- App.js                <= Routes setup
```

TodoContext.js

```
// context/TodoContext.js
import { createContext, useReducer, useContext } from 'react';

const TodoContext = createContext();

const initialState = { todos: [], filter: 'all' };

function todoReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return {
        ...state,
        todos: [...state.todos, {
          id: Date.now(), text: action.payload, done: false
        }]
      };
    case 'TOGGLE':
      return {
        ...state,
        todos: state.todos.map(t =>
          t.id === action.payload ? { ...t, done: !t.done } : t
        )
      };
    case 'DELETE':
      return { ...state, todos: state.todos.filter(t => t.id !== action.payload) };
    case 'SET_FILTER':
      return { ...state, filter: action.payload };
    default:
      return state;
  }
}
```

```
export function TodoProvider({ children }) {
  const [state, dispatch] = useReducer(todoReducer, initialState);
  return (
    <TodoContext.Provider value={{ state, dispatch }}>
      {children}
    </TodoContext.Provider>
  );
}

export const useTodoContext = () => useContext(TodoContext);
```

App.js

```
// App.js
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { TodoProvider } from '../context/TodoContext';
import Navbar from '../components/Navbar';
import Home from '../pages/Home';
import About from '../pages/About';

function App() {
  return (
    <TodoProvider>
      <BrowserRouter>
        <Navbar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
        </Routes>
      </BrowserRouter>
    </TodoProvider>
  );
}

export default App;
```

pages/Home.js

```
// pages/Home.js
import { useState } from 'react';
import { useTodoContext } from '../context/TodoContext';

function Home() {
  const { state, dispatch } = useTodoContext();
  const [text, setText] = useState('');

  const addTodo = () => {
    if (text.trim()) {
      dispatch({ type: 'ADD', payload: text.trim() });
      setText('');
    }
  };

  const filtered = state.todos.filter(t => {
    if (state.filter === 'active') return !t.done;
    if (state.filter === 'done') return t.done;
    return true;
  });
```

```
});

return (
  <div>
    <h1>Todo App</h1>
    <div>
      <input value={text} onChange={e => setText(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && addTodo()}
        placeholder="Add a todo..." />
      <button onClick={addTodo}>Add</button>
    </div>
    <div>
      {[ 'all', 'active', 'done' ].map(f => (
        <button key={f} onClick={() => dispatch({ type: 'SET_FILTER', payload: f })}>
          {f}
        </button>
      ))}
    </div>
    <ul>
      {filtered.map(todo => (
        <li key={todo.id}>
          <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>
            {todo.text}
          </span>
          <button onClick={() => dispatch({ type: 'TOGGLE', payload: todo.id })}>
            {todo.done ? 'Undo' : 'Done'}
          </button>
          <button onClick={() => dispatch({ type: 'DELETE', payload: todo.id })}>
            Delete
          </button>
        </li>
      ))}
    </ul>
  </div>
);
}

export default Home;
```

■ 11. Day 2 Quick Reference Cheat Sheet

Hooks Cheat Sheet

Hook	Import	Key Usage
useEffect	from 'react'	useEffect(() => { ... }, [deps])
useRef	from 'react'	const ref = useRef(null)
useContext	from 'react'	const val = useContext(MyContext)
useReducer	from 'react'	const [s, dispatch] = useReducer(fn, init)
useMemo	from 'react'	const v = useMemo(() => calc, [deps])
useCallback	from 'react'	const fn = useCallback(() => {}, [deps])
createContext	from 'react'	const Ctx = createContext(defaultValue)

React Router v6 Cheat Sheet

Hook / Component	Usage	Notes
BrowserRouter	Wrap entire app	Once, at root
Routes + Route	path="/" element={}	Replaces Switch
Link	to="/path"	No page reload
NavLink	style={({isActive}) => ...}	Auto active class
Outlet	Inside layout component	Renders child routes
useNavigate	navigate("/path")	Programmatic nav
useParams	const { id } = useParams()	URL :params
useLocation	const { pathname } = useLocation()	Current URL info
useSearchParams	const [sp, set] = useSearchParams()	Query strings

useEffect Dependency Array Quick Guide

Scenario	Dependency Array	Effect Runs
Run once on mount	[]	After first render only
Run on prop/state change	[value]	When value changes
Run on every render	omitted	After every render
Cleanup on unmount	return () => {}	Before component is removed

When to Use Which Hook

Situation	Best Hook
Simple toggle / counter	useState
Side effects: fetch, subscriptions	useEffect
Access DOM element directly	useRef
Share data across many components	useContext
Complex state with many transitions	useReducer
Expensive computation in render	useMemo
Stable callback for child components	useCallback
Reusable stateful logic	Custom Hook
